

11.3 PARTIAL-ORDER PLANNING

Forward and backward state-space search are particular forms of *totally ordered* plan search. They explore only strictly linear sequences of actions directly connected to the start or goal. This means that they cannot take advantage of problem decomposition. Rather than work on each subproblem separately, they must always make decisions about how to sequence actions from all the subproblems. We would prefer an approach that works on several subgoals independently, solves them with several subplans, and then combines the subplans.

Such an approach also has the advantage of flexibility in the order in which it *constructs* the plan. That is, the planner can work on “obvious” or “important” decisions first, rather than being forced to work on steps in chronological order. For example, a planning agent that is in Berkeley and wishes to be in Monte Carlo might first try to find a flight from San Francisco to Paris; given information about the departure and arrival times, it can then work on ways to get to and from the airports.

LEAST COMMITMENT

The general strategy of delaying a choice during search is called a **least commitment** strategy. There is no formal definition of least commitment, and clearly some degree of commitment is necessary, lest the search would make no progress. Despite the informality, least commitment is a useful concept for analyzing when decisions should be made in any search problem.

⁵ Technically, STRIPS-style planning is PSPACE-complete unless actions have only positive preconditions and only one effect literal (Bylander, 1994).

Our first concrete example will be much simpler than planning a vacation. Consider the simple problem of putting on a pair of shoes. We can describe this as a formal planning problem as follows:

```
Goal(RightShoeOn  $\wedge$  LeftShoeOn)
Init()
Action(RightShoe, PRECOND:RightSockOn, EFFECT:RightShoeOn)
Action(RightSock, EFFECT:RightSockOn)
Action(LeftShoe, PRECOND:LeftSockOn, EFFECT:LeftShoeOn)
Action(LeftSock, EFFECT:LeftSockOn) .
```

A planner should be able to come up with the two-action sequence *RightSock* followed by *RightShoe* to achieve the first conjunct of the goal and the sequence *LeftSock* followed by *LeftShoe* for the second conjunct. Then the two sequences can be combined to yield the final plan. In doing this, the planner will be manipulating the two subsequences independently, without committing to whether an action in one sequence is before or after an action in the other. Any planning algorithm that can place two actions into a plan without specifying which comes first is called a **partial-order planner**. Figure 11.6 shows the partial-order plan that is the solution to the shoes and socks problem. Note that the solution is represented as a *graph* of actions, not a sequence. Note also the “dummy” actions called *Start* and *Finish*, which mark the beginning and end of the plan. Calling them actions simplifies things, because now every step of a plan is an action. The partial-order solution corresponds to six possible total-order plans; each of these is called a **linearization** of the partial-order plan.

PARTIAL-ORDER
PLANNER

LINEARIZATION

Partial-order planning can be implemented as a search in the space of partial-order plans. (From now on, we will just call them “plans.”) That is, we start with an empty plan. Then we consider ways of refining the plan until we come up with a complete plan that solves the problem. The actions in this search are not actions in the world, but actions on plans: adding a step to the plan, imposing an ordering that puts one action before another, and so on.

We will define the POP algorithm for partial-order planning. It is traditional to write out the POP algorithm as a stand-alone program, but we will instead formulate partial-order planning as an instance of a search problem. This allows us to focus on the plan refinement steps that can be applied, rather than worrying about how the algorithm explores the space. In fact, a wide variety of uninformed or heuristic search methods can be applied once the search problem is formulated.

Remember that the states of our search problem will be (mostly unfinished) plans. To avoid confusion with the states of the world, we will talk about plans rather than states. Each plan has the following four components, where the first two define the steps of the plan and the last two serve a bookkeeping function to determine how plans can be extended:

- A set of **actions** that make up the steps of the plan. These are taken from the set of actions in the planning problem. The “empty” plan contains just the *Start* and *Finish* actions. *Start* has no preconditions and has as its effect all the literals in the initial state of the planning problem. *Finish* has no effects and has as its preconditions the goal literals of the planning problem.

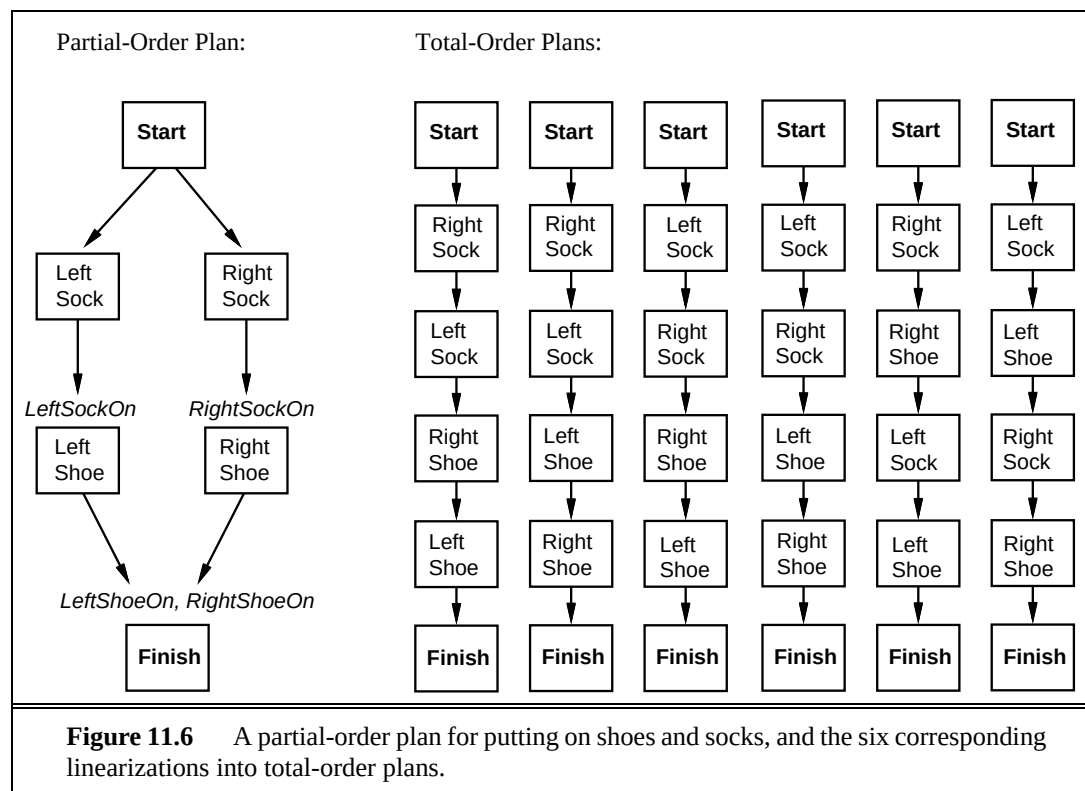


Figure 11.6 A partial-order plan for putting on shoes and socks, and the six corresponding linearizations into total-order plans.

ORDERING
CONSTRAINTS

- A set of **ordering constraints**. Each ordering constraint is of the form $A \prec B$, which is read as “ A before B ” and means that action A must be executed sometime before action B , but not necessarily immediately before. The ordering constraints must describe a proper partial order. Any cycle—such as $A \prec B$ and $B \prec A$ —represents a contradiction, so an ordering constraint cannot be added to the plan if it creates a cycle.

CAUSAL LINKS

ACHIEVES

- A set of **causal links**. A causal link between two actions A and B in the plan is written as $A \xrightarrow{p} B$ and is read as “ A achieves p for B .” For example, the causal link

$$\text{RightSock} \xrightarrow{\text{RightSockOn}} \text{RightShoe}$$

asserts that *RightSockOn* is an effect of the *RightSock* action and a precondition of *RightShoe*. It also asserts that *RightSockOn* must remain true from the time of action *RightSock* to the time of action *RightShoe*. In other words, the plan may not be extended by adding a new action C that **conflicts** with the causal link. An action C conflicts with $A \xrightarrow{p} B$ if C has the effect $\neg p$ and if C could (according to the ordering constraints) come after A and before B . Some authors call causal links **protection intervals**, because the link $A \xrightarrow{p} B$ protects p from being negated over the interval from A to B .

CONFLICTS

OPEN
PRECONDITIONS

- A set of **open preconditions**. A precondition is open if it is not achieved by some action in the plan. Planners will work to reduce the set of open preconditions to the empty set, without introducing a contradiction.

For example, the final plan in Figure 11.6 has the following components (not shown are the ordering constraints that put every other action after *Start* and before *Finish*):

Actions: $\{RightSock, RightShoe, LeftSock, LeftShoe, Start, Finish\}$
 Orderings: $\{RightSock \prec RightShoe, LeftSock \prec LeftShoe\}$
 Links: $\{RightSock \xrightarrow{RightSockOn} RightShoe, LeftSock \xrightarrow{LeftSockOn} LeftShoe, RightShoe \xrightarrow{RightShoeOn} Finish, LeftShoe \xrightarrow{LeftShoeOn} Finish\}$
 Open Preconditions: $\{\}$.

CONSISTENT PLAN



We define a **consistent plan** as a plan in which there are no cycles in the ordering constraints and no conflicts with the causal links. A consistent plan with no open preconditions is a **solution**. A moment's thought should convince the reader of the following fact: *every linearization of a partial-order solution is a total-order solution whose execution from the initial state will reach a goal state*. This means that we can extend the notion of “executing a plan” from total-order to partial-order plans. A partial-order plan is executed by repeatedly choosing *any* of the possible next actions. We will see in Chapter 12 that the flexibility available to the agent as it executes the plan can be very useful when the world fails to cooperate. The flexible ordering also makes it easier to combine smaller plans into larger ones, because each of the small plans can reorder its actions to avoid conflict with the other plans.

Now we are ready to formulate the search problem that POP solves. We will begin with a formulation suitable for propositional planning problems, leaving the first-order complications for later. As usual, the definition includes the initial state, actions, and goal test.

- The initial plan contains *Start* and *Finish*, the ordering constraint $Start \prec Finish$, and no causal links and has all the preconditions in *Finish* as open preconditions.
- The successor function arbitrarily picks one open precondition p on an action B and generates a successor plan for every possible consistent way of choosing an action A that achieves p . Consistency is enforced as follows:
 1. The causal link $A \xrightarrow{p} B$ and the ordering constraint $A \prec B$ are added to the plan. Action A may be an existing action in the plan or a new one. If it is new, add it to the plan and also add $Start \prec A$ and $A \prec Finish$.
 2. We resolve conflicts between the new causal link and all existing actions and between the action A (if it is new) and all existing causal links. A conflict between $A \xrightarrow{p} B$ and C is resolved by making C occur at some time outside the protection interval, either by adding $B \prec C$ or $C \prec A$. We add successor states for either or both if they result in consistent plans.
- The goal test checks whether a plan is a solution to the original planning problem. Because only consistent plans are generated, the goal test just needs to check that there are no open preconditions.

Remember that the actions considered by the search algorithms under this formulation are plan refinement steps rather than the real actions from the domain itself. The path cost is therefore irrelevant, strictly speaking, because the only thing that matters is the total cost of the real actions in the plan to which the path leads. Nonetheless, it is possible to specify a path cost function that reflects the real plan costs: we charge 1 for each real action added to

the plan and 0 for all other refinement steps. In this way, $g(n)$, where n is a plan, will be equal to the number of real actions in the plan. A heuristic estimate $h(n)$ can also be used.

At first glance, one might think that the successor function should include successors for every open p , not just for one of them. This would be redundant and inefficient, however, for the same reason that constraint satisfaction algorithms don't include successors for every possible variable: the order in which we consider open preconditions (like the order in which we consider CSP variables) is commutative. (See page 141.) Thus, we can choose an arbitrary ordering and still have a complete algorithm. Choosing the right ordering can lead to a faster search, but all orderings end up with the same set of candidate solutions.

A partial-order planning example

Now let's look at how POP solves the spare tire problem from Section 11.1. The problem description is repeated in Figure 11.7.

```

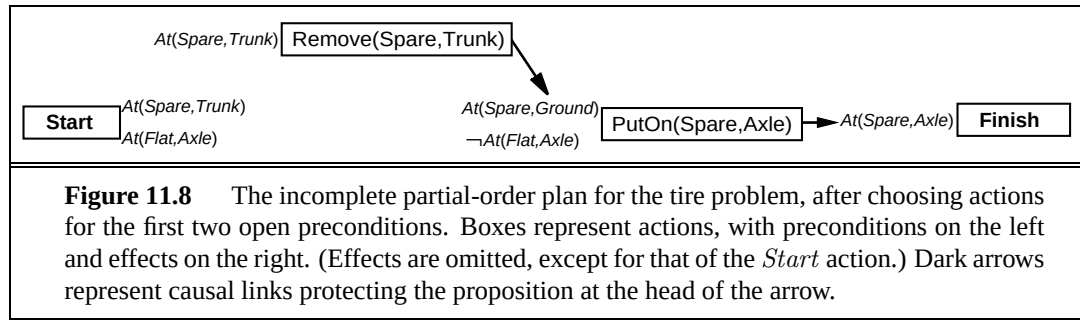
Init( $At(Flat, Axle) \wedge At(Spare, Trunk)$ )
Goal( $At(Spare, Axle)$ )
Action( $Remove(Spare, Trunk)$ ,
  PRECOND:  $At(Spare, Trunk)$ 
  EFFECT:  $\neg At(Spare, Trunk) \wedge At(Spare, Ground)$ )
Action( $Remove(Flat, Axle)$ ,
  PRECOND:  $At(Flat, Axle)$ 
  EFFECT:  $\neg At(Flat, Axle) \wedge At(Flat, Ground)$ )
Action( $PutOn(Spare, Axle)$ ,
  PRECOND:  $At(Spare, Ground) \wedge \neg At(Flat, Axle)$ 
  EFFECT:  $\neg At(Spare, Ground) \wedge At(Spare, Axle)$ )
Action( $LeaveOvernight$ ,
  PRECOND:
  EFFECT:  $\neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk)$ 
          $\wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle)$ )

```

Figure 11.7 The simple flat tire problem description.

The search for a solution begins with the initial plan, containing a *Start* action with the effect $At(Spare, Trunk) \wedge At(Flat, Axle)$ and a *Finish* action with the sole precondition $At(Spare, Axle)$. Then we generate successors by picking an open precondition to work on (irrevocably) and choosing among the possible actions to achieve it. For now, we will not worry about a heuristic function to help with these decisions; we will make seemingly arbitrary choices. The sequence of events is as follows:

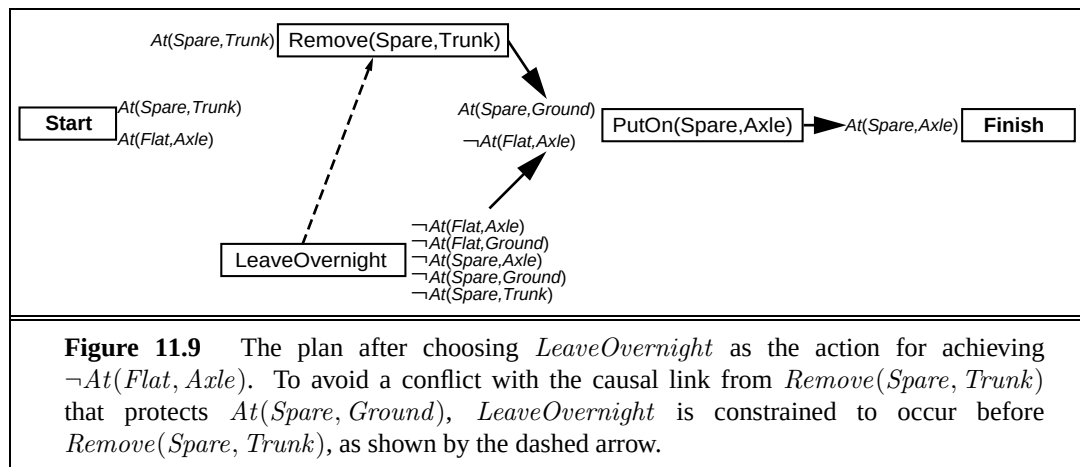
1. Pick the only open precondition, $At(Spare, Axle)$ of *Finish*. Choose the only applicable action, $PutOn(Spare, Axle)$.
2. Pick the $At(Spare, Ground)$ precondition of $PutOn(Spare, Axle)$. Choose the only applicable action, $Remove(Spare, Trunk)$ to achieve it. The resulting plan is shown in Figure 11.8.



3. Pick the $\neg At(Flat, Axle)$ precondition of *PutOn(Spare, Axle)*. Just to be contrary, choose the *LeaveOvernight* action rather than the *Remove(Flat, Axle)* action. Notice that *LeaveOvernight* also has the effect $\neg At(Spare, Ground)$, which means it conflicts with the causal link

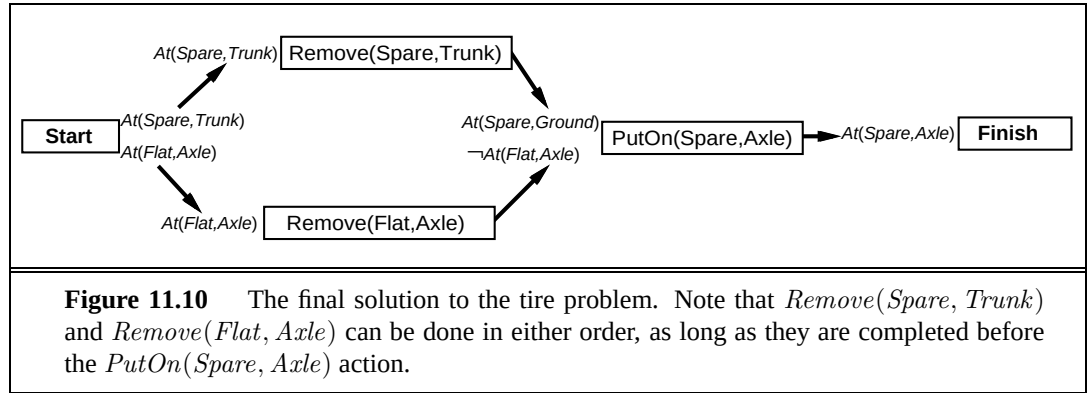
$$Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle).$$

To resolve the conflict we add an ordering constraint putting *LeaveOvernight* before *Remove(Spare, Trunk)*. The resulting plan is shown in Figure 11.9. (Why does this resolve the conflict, and why is there no other way to resolve it?)



4. The only remaining open precondition at this point is the *At(Spare, Trunk)* precondition of the action *Remove(Spare, Trunk)*. The only action that can achieve it is the existing *Start* action, but the causal link from *Start* to *Remove(Spare, Trunk)* is in conflict with the $\neg At(Spare, Trunk)$ effect of *LeaveOvernight*. This time there is no way to resolve the conflict with *LeaveOvernight*: we cannot order it before *Start* (because nothing can come before *Start*), and we cannot order it after *Remove(Spare, Trunk)* (because there is already a constraint ordering it before *Remove(Spare, Trunk)*). So we are forced to back up, remove the *Remove(Spare, Trunk)* action and the last two causal links, and return to the state in Figure 11.8. In essence, the planner has proved that *LeaveOvernight* doesn't work as a way to change a tire.

5. Consider again the $\neg At(Flat, Axle)$ precondition of $PutOn(Spare, Axle)$. This time, we choose $Remove(Flat, Axle)$.
6. Once again, pick the $At(Spare, Tire)$ precondition of $Remove(Spare, Trunk)$ and choose $Start$ to achieve it. This time there are no conflicts.
7. Pick the $At(Flat, Axle)$ precondition of $Remove(Flat, Axle)$, and choose $Start$ to achieve it. This gives us a complete, consistent plan—in other words a solution—as shown in Figure 11.10.



Although this example is very simple, it illustrates some of the strengths of partial-order planning. First, the causal links lead to early pruning of portions of the search space that, because of irresolvable conflicts, contain no solutions. Second, the solution in Figure 11.10 is a partial-order plan. In this case the advantage is small, because there are only two possible linearizations; nonetheless, an agent might welcome the flexibility—for example, if the tire has to be changed in the middle of heavy traffic.

The example also points to some possible improvements that could be made. For example, there is duplication of effort: *Start* is linked to $Remove(Spare, Trunk)$ before the conflict causes a backtrack and is then unlinked by backtracking even though it is not involved in the conflict. It is then relinked as the search continues. This is typical of chronological backtracking and might be mitigated by dependency-directed backtracking.

Partial-order planning with unbound variables

In this section, we consider the complications that can arise when POP is used with first-order action representations that include variables. Suppose we have a blocks world problem (Figure 11.4) with the open precondition $On(A, B)$ and the action

$Action(Move(b, x, y),$
 $\text{PRECOND: } On(b, x) \wedge Clear(b) \wedge Clear(y),$
 $\text{EFFECT: } On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y)) .$

This action achieves $On(A, B)$ because the effect $On(b, y)$ unifies with $On(A, B)$ with the substitution $\{b/A, y/B\}$. We then apply this substitution to the action, yielding

$Action(Move(A, x, B),$
 PRECOND: $On(A, x) \wedge Clear(A) \wedge Clear(B),$
 EFFECT: $On(A, B) \wedge Clear(x) \wedge \neg On(A, x) \wedge \neg Clear(B))$.

This leaves the variable x unbound. That is, the action says to move block A from *somewhere*, without yet saying whence. This is another example of the least commitment principle: we can delay making the choice until some other step in the plan makes it for us. For example, suppose we have $On(A, D)$ in the initial state. Then the *Start* action can be used to achieve $On(A, x)$, binding x to D . This strategy of waiting for more information before choosing x is often more efficient than trying every possible value of x and backtracking for each one that fails.

The presence of variables in preconditions and actions complicates the process of detecting and resolving conflicts. For example, when $Move(A, x, B)$ is added to the plan, we will need a causal link

$Move(A, x, B) \xrightarrow{On(A, B)} Finish$.

If there is another action M_2 with effect $\neg On(A, z)$, then M_2 conflicts only if z is B . To accommodate this possibility, we extend the representation of plans to include a set of **inequality constraints** of the form $z \neq X$ where z is a variable and X is either another variable or a constant symbol. In this case, we would resolve the conflict by adding $z \neq B$, which means that future extensions to the plan can instantiate z to any value except B . Anytime we apply a substitution to a plan, we must check that the inequalities do not contradict the substitution. For example, a substitution that includes x/y conflicts with the inequality constraint $x \neq y$. Such conflicts cannot be resolved, so the planner must backtrack.

A more extensive example of POP planning with variables in the blocks world is given in Section 12.6.

Heuristics for partial-order planning

Compared with total-order planning, partial-order planning has a clear advantage in being able to decompose problems into subproblems. It also has a disadvantage in that it does not represent states directly, so it is harder to estimate how far a partial-order plan is from achieving a goal. At present, there is less understanding of how to compute accurate heuristics for partial-order planning than for total-order planning.

The most obvious heuristic is to count the number of distinct open preconditions. This can be improved by subtracting the number of open preconditions that match literals in the *Start* state. As in the total-order case, this overestimates the cost when there are actions that achieve multiple goals and underestimates the cost when there are negative interactions between plan steps. The next section presents an approach that allows us to get much more accurate heuristics from a relaxed problem.

The heuristic function is used to choose which plan to refine. Given this choice, the algorithm generates successors based on the selection of a single open precondition to work

on. As in the case of variable selection on constraint satisfaction algorithms, this selection has a large impact on efficiency. The **most-constrained-variable** heuristic from CSPs can be adapted for planning algorithms and seems to work well. The idea is to select the open condition that can be satisfied in the *fewest* number of ways. There are two special cases of this heuristic. First, if an open condition cannot be achieved by any action, the heuristic will select it; this is a good idea because early detection of impossibility can save a great deal of work. Second, if an open condition can be achieved in only one way, then it should be selected because the decision is unavoidable and could provide additional constraints on other choices still to be made. Although full computation of the number of ways to satisfy each open condition is expensive and not always worthwhile, experiments show that handling the two special cases provides very substantial speedups.

11.4 PLANNING GRAPHS

PLANNING GRAPH

All of the heuristics we have suggested for total-order and partial-order planning can suffer from inaccuracies. This section shows how a special data structure called a **planning graph** can be used to give better heuristic estimates. These heuristics can be applied to any of the search techniques we have seen so far. Alternatively, we can extract a solution directly from the planning graph, using a specialized algorithm such as the one called GRAPHPLAN.

LEVELS

A planning graph consists of a sequence of **levels** that correspond to time steps in the plan, where level 0 is the initial state. Each level contains a set of literals and a set of actions. Roughly speaking, the literals are all those that *could* be true at that time step, depending on the actions executed at preceding time steps. Also roughly speaking, the actions are all those actions that *could* have their preconditions satisfied at that time step, depending on which of the literals actually hold. We say “roughly speaking” because the planning graph records only a restricted subset of the possible negative interactions among actions; therefore, it might be optimistic about the minimum number of time steps required for a literal to become true. Nonetheless, this number of steps in the planning graph provides a good estimate of how difficult it is to achieve a given literal from the initial state. More importantly, the planning graph is defined in such a way that it can be constructed very efficiently.

Planning graphs work only for propositional planning problems—ones with no variables. As we mentioned in Section 11.1, both STRIPS and ADL representations can be propositionalized. For problems with large numbers of objects, this could result in a very substantial blowup in the number of action schemata. Despite this, planning graphs have proved to be effective tools for solving hard planning problems.

We will illustrate planning graphs with a simple example. (More complex examples lead to graphs that won’t fit on the page.) Figure 11.11 shows a problem, and Figure 11.12 shows its planning graph. We start with state level S_0 , which represents the problem’s initial state. We follow that with action level A_0 , in which we place all the actions whose preconditions are satisfied in the previous level. Each action is connected to its preconditions in S_0 and its effects in S_1 , in this case introducing new literals into S_1 that were not in S_0 .

```

Init(Have(Cake))
Goal(Have(Cake) ∧ Eaten(Cake))
Action(Eat(Cake))
  PRECOND: Have(Cake)
  EFFECT: ¬ Have(Cake) ∧ Eaten(Cake)
Action(Bake(Cake))
  PRECOND: ¬ Have(Cake)
  EFFECT: Have(Cake)

```

Figure 11.11 The “have cake and eat cake too” problem.

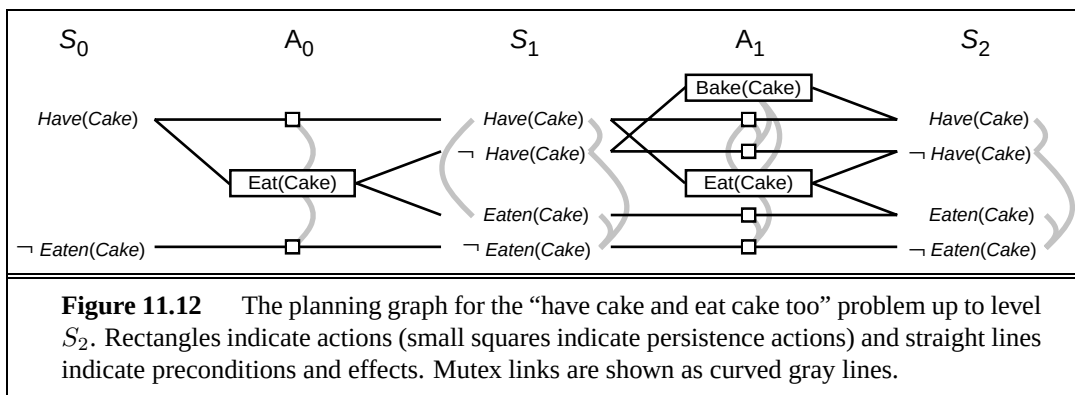


Figure 11.12 The planning graph for the “have cake and eat cake too” problem up to level S_2 . Rectangles indicate actions (small squares indicate persistence actions) and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines.

The planning graph needs a way to represent inaction as well as action. That is, it needs the equivalent of the frame axioms in situation calculus that allow a literal to remain true from one situation to the next if no action alters it. In a planning graph this is done with a set of **persistence actions**. For every positive and negative literal C , we add to the problem a persistence action with precondition C and effect C . Figure 11.12 shows one “real” action, $Eat(Cake)$ in A_0 , along with two persistence actions drawn as small square boxes.

Level A_0 contains all the actions that *could* occur in state S_0 , but just as importantly it records conflicts between actions that would prevent them from occurring together. The gray lines in Figure 11.12 indicate **mutual exclusion** (or **mutex**) links. For example, $Eat(Cake)$ is mutually exclusive with the persistence of either $Have(Cake)$ or $\neg Eaten(Cake)$. We shall see shortly how mutex links are computed.

Level S_1 contains all the literals that could result from picking any subset of the actions in A_0 . It also contains mutex links (gray lines) indicating literals that could not appear together, regardless of the choice of actions. For example, $Have(Cake)$ and $Eaten(Cake)$ are mutex: depending on the choice of actions in A_0 , one or the other, but not both, could be the result. In other words, S_1 represents multiple states, just as regression state-space search does, and the mutex links are constraints that define the set of possible states.

We continue in this way, alternating between state level S_i and action level A_i until we reach a level where two consecutive levels are identical. At this point, we say that the graph

PERSISTENCE
ACTIONS

MUTUAL EXCLUSION
MUTEX

LEVELED OFF

has **leveled off**. Every subsequent level will be identical, so further expansion is unnecessary.

What we end up with is a structure where every A_i level contains all the actions that are applicable in S_i , along with constraints saying which pairs of actions cannot both be executed. Every S_i level contains all the literals that could result from any possible choice of actions in A_{i-1} , along with constraints saying which pairs of literals are not possible. It is important to note that the process of constructing the planning graph does *not* require choosing among actions, which would entail combinatorial search. Instead, it just records the impossibility of certain choices using mutex links. The complexity of constructing the planning graph is a low-order polynomial in the number of actions and literals, whereas the state space is exponential in the number of literals.

We now define mutex links for both actions and literals. A mutex relation holds between two *actions* at a given level if any of the following three conditions holds:

- *Inconsistent effects*: one action negates an effect of the other. For example $Eat(Cake)$ and the persistence of $Have(Cake)$ have inconsistent effects because they disagree on the effect $Have(Cake)$.
- *Interference*: one of the effects of one action is the negation of a precondition of the other. For example $Eat(Cake)$ interferes with the persistence of $Have(Cake)$ by negating its precondition.
- *Competing needs*: one of the preconditions of one action is mutually exclusive with a precondition of the other. For example, $Bake(Cake)$ and $Eat(Cake)$ are mutex because they compete on the value of the $Have(Cake)$ precondition.

A mutex relation holds between two *literals* at the same level if one is the negation of the other or if each possible pair of actions that could achieve the two literals is mutually exclusive. This condition is called *inconsistent support*. For example, $Have(Cake)$ and $Eaten(Cake)$ are mutex in S_1 because the only way of achieving $Have(Cake)$, the persistence action, is mutex with the only way of achieving $Eaten(Cake)$, namely $Eat(Cake)$. In S_2 the two literals are not mutex because there are new ways of achieving them, such as $Bake(Cake)$ and the persistence of $Eaten(Cake)$, that are not mutex.

Planning graphs for heuristic estimation



A planning graph, once constructed, is a rich source of information about the problem. For example, *a literal that does not appear in the final level of the graph cannot be achieved by any plan*. This observation can be used in backward search as follows: any state containing an unachievable literal has a cost $h(n) = \infty$. Similarly, in partial-order planning, any plan with an unachievable open condition has $h(n) = \infty$.

This idea can be made more general. We can estimate the cost of achieving any goal literal as the level at which it first appears in the planning graph. We will call this the **level cost** of the goal. In Figure 11.12, $Have(Cake)$ has level cost 0 and $Eaten(Cake)$ has level cost 1. It is easy to show (Exercise 11.9) that these estimates are admissible for the individual goals. The estimate might not be very good, however, because planning graphs allow several actions at each level whereas the heuristic counts just the level and not the number of actions. For this reason, it is common to use a **serial planning graph** for computing heuristics. A

LEVEL COST

SERIAL PLANNING
GRAPH

serial graph insists that only one action can actually occur at any given time step; this is done by adding mutex links between every pair of actions except persistence actions. Level costs extracted from serial graphs are often quite reasonable estimates of actual costs.

MAX-LEVEL

LEVEL SUM

SET-LEVEL

To estimate the cost of a conjunction of goals, there are three simple approaches. The **max-level** heuristic simply takes the maximum level cost of any of the goals; this is admissible, but not necessarily very accurate. The **level sum** heuristic, following the subgoal independence assumption, returns the sum of the level costs of the goals; this is inadmissible but works very well in practice for problems that are largely decomposable. It is much more accurate than the number-of-unsatisfied-goals heuristic from Section 11.2. For our problem, the heuristic estimate for the conjunctive goal $Have(Cake) \wedge Eaten(Cake)$ will be $0 + 1 = 1$, whereas the correct answer is 2. Moreover, if we eliminated the $Bake(Cake)$ action, the estimate would still be 1, but the conjunctive goal would be impossible. Finally, the **set-level** heuristic finds the level at which all the literals in the conjunctive goal appear in the planning graph without any pair of them being mutually exclusive. This heuristic gives the correct values of 2 for our original problem and infinity for the problem without $Bake(Cake)$. It dominates the max-level heuristic and works extremely well on tasks in which there is a good deal of interaction among subplans.

As a tool for generating accurate heuristics, we can view the planning graph as a relaxed problem that is efficiently soluble. To understand the nature of the relaxed problem, we need to understand exactly what it means for a literal g to appear at level S_i in the planning graph. Ideally, we would like it to be a guarantee that there exists a plan with i action levels that achieves g , and also that if g does not appear that there is no such plan. Unfortunately, making that guarantee is as difficult as solving the original planning problem. So the planning graph makes the second half of the guarantee (if g does not appear, there is no plan), but if g does appear, then all the planning graph promises is that there is a plan that *possibly* achieves g and has no “obvious” flaws. An obvious flaw is defined as a flaw that can be detected by considering two actions or two literals at a time—in other words, by looking at the mutex relations. There could be more subtle flaws involving three, four, or more actions, but experience has shown that it is not worth the computational effort to keep track of these possible flaws. This is similar to the lesson learned from constraint satisfaction problems that it is often worthwhile to compute 2-consistency before searching for a solution, but less often worthwhile to compute 3-consistency or higher. (See Section 5.2.)

The GRAPHPLAN algorithm

This subsection shows how to extract a plan directly from the planning graph, rather than just using the graph to provide a heuristic. The GRAPHPLAN algorithm (Figure 11.13) has two main steps, which alternate within a loop. First, it checks whether all the goal literals are present in the current level with no mutex links between any pair of them. If this is the case, then a solution *might* exist within the current graph, so the algorithm tries to extract that solution. Otherwise, it expands the graph by adding the actions for the current level and the state literals for the next level. The process continues until either a solution is found or it is learned that no solution exists.

```

function GRAPHPLAN(problem) returns solution or failure

  graph  $\leftarrow$  INITIAL-PLANNING-GRAPH(problem)
  goals  $\leftarrow$  GOALS[problem]
  loop do
    if goals all non-mutex in last level of graph then do
      solution  $\leftarrow$  EXTRACT-SOLUTION(graph, goals, LENGTH(graph))
      if solution  $\neq$  failure then return solution
      else if NO-SOLUTION-POSSIBLE(graph) then return failure
    graph  $\leftarrow$  EXPAND-GRAPH(graph, problem)

```

Figure 11.13 The GRAPHPLAN algorithm. GRAPHPLAN alternates between a solution extraction step and a graph expansion step. EXTRACT-SOLUTION looks for whether a plan can be found, starting at the end and searching backwards. EXPAND-GRAPH adds the actions whose preconditions exist at level S_0 (i.e., all the actions except $PutOn(Spare, Axle)$), along with persistence actions for all the literals in S_0 . The effects of the actions are added at level S_1 . EXPAND-GRAPH then looks for mutex relations and adds them to the graph.

Let us now trace the operation of GRAPHPLAN on the spare tire problem from Section 11.1. The entire graph is shown in Figure 11.14. The first line of GRAPHPLAN initializes the planning graph to a one-level (S_0) graph consisting of the five literals from the initial state. The goal literal $At(Spare, Axle)$ is not present in S_0 , so we need not call EXTRACT-SOLUTION—we are certain that there is no solution yet. Instead, EXPAND-GRAPH adds the three actions whose preconditions exist at level S_0 (i.e., all the actions except $PutOn(Spare, Axle)$), along with persistence actions for all the literals in S_0 . The effects of the actions are added at level S_1 . EXPAND-GRAPH then looks for mutex relations and adds them to the graph.

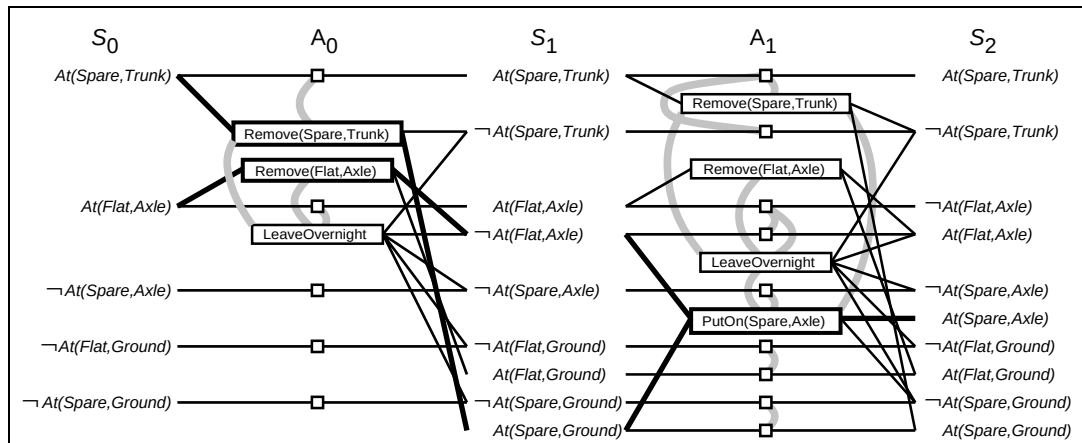


Figure 11.14 The planning graph for the spare tire problem after expansion to level S_2 . Mutex links are shown as gray lines. Only some representative mutexes are shown, because the graph would be too cluttered if we showed them all. The solution is indicated by bold lines and outlines.

$At(Spare, Axle)$ is still not present in S_1 , so again we do not call EXTRACT-SOLUTION. The call to EXPAND-GRAPH gives us the planning graph shown in Figure 11.14. Now that we have the full complement of actions, it is worthwhile to look at some of the examples of mutex relations and their causes:

- *Inconsistent effects:* $Remove(Spare, Trunk)$ is mutex with $LeaveOvernight$ because one has the effect $At(Spare, Ground)$ and the other has its negation.
- *Interference:* $Remove(Flat, Axle)$ is mutex with $LeaveOvernight$ because one has the precondition $At(Flat, Axle)$ and the other has its negation as an effect.
- *Competing needs:* $PutOn(Spare, Axle)$ is mutex with $Remove(Flat, Axle)$ because one has $At(Flat, Axle)$ as a precondition and the other has its negation.
- *Inconsistent support:* $At(Spare, Axle)$ is mutex with $At(Flat, Axle)$ in S_2 because the only way of achieving $At(Spare, Axle)$ is by $PutOn(Spare, Axle)$, and that is mutex with the persistence action that is the only way of achieving $At(Flat, Axle)$. Thus, the mutex relations detect the immediate conflict that arises from trying to put two objects in the same place at the same time.

This time, when we go back to the start of the loop, all the literals from the goal are present in S_2 , and none of them is mutex with any other. That means that a solution might exist, and EXTRACT-SOLUTION will try to find it. In essence, EXTRACT-SOLUTION solves a Boolean CSP whose variables are the actions at each level, and the values for each variable are *in* or *out* of the plan. We can use standard CSP algorithms for this, or we can define EXTRACT-SOLUTION as a search problem, where each state in the search contains a pointer to a level in the planning graph and a set of unsatisfied goals. We define this search problem as follows:

- The initial state is the last level of the planning graph, S_n , along with the set of goals from the planning problem.
- The actions available in a state at level S_i are to select any conflict-free subset of the actions in A_{i-1} whose effects cover the goals in the state. The resulting state has level S_{i-1} and has as its set of goals the preconditions for the selected set of actions. By “conflict-free,” we mean a set of actions such that no two of them are mutex, and no two of their preconditions are mutex.
- The goal is to reach a state at level S_0 such that all the goals are satisfied.
- The cost of each action is 1.

For this particular problem, we start at S_2 with the goal $At(Spare, Axle)$. The only choice we have for achieving the goal set is $PutOn(Spare, Axle)$. That brings us to a search state at S_1 with goals $At(Spare, Ground)$ and $\neg At(Flat, Axle)$. The former can be achieved only by $Remove(Spare, Trunk)$, and the latter by either $Remove(Flat, Axle)$ or $LeaveOvernight$. But $LeaveOvernight$ is mutex with $Remove(Spare, Trunk)$, so the only solution is to choose $Remove(Spare, Trunk)$ and $Remove(Flat, Axle)$. That brings us to a search state at S_0 with the goals $At(Spare, Trunk)$ and $At(Flat, Axle)$. Both of these are present in the state, so we have a solution: the actions $Remove(Spare, Trunk)$ and $Remove(Flat, Axle)$ in level A_0 , followed by $PutOn(Spare, Axle)$ in A_1 .

We know that planning is PSPACE-complete and that constructing the planning graph takes polynomial time, so it must be the case that solution extraction is intractable in the worst case. Therefore, we will need some heuristic guidance for choosing among actions during the backward search. One approach that works well in practice is a greedy algorithm based on the level cost of the literals. For any set of goals, we proceed in the following order:

1. Pick first the literal with the highest level cost.
2. To achieve that literal, choose the action with the easiest preconditions first. That is, choose an action such that the sum (or maximum) of the level costs of its preconditions is smallest.

Termination of GRAPHPLAN

So far, we have skated over the question of termination. If a problem has no solution, can we be sure that GRAPHPLAN will not loop forever, extending the planning graph at each iteration? The answer is yes, but the full proof is beyond the scope of this book. Here, we outline just the main ideas, particularly the ones that shed light on planning graphs in general.

The first step is to notice that certain properties of planning graphs are monotonically increasing or decreasing. “X increases monotonically” means that the set of Xs at level $i + 1$ is a superset (not necessarily proper) of the set at level i . The properties are as follows:

- *Literals increase monotonically*: Once a literal appears at a given level, it will appear at all subsequent levels. This is because of the persistence actions; once a literal shows up, persistence actions cause it to stay forever.
- *Actions increase monotonically*: Once an action appears at a given level, it will appear at all subsequent levels. This is a consequence of literals’ increasing; if the preconditions of an action appear at one level, they will appear at subsequent levels, and thus so will the action.
- *Mutexes decrease monotonically*: If two actions are mutex at a given level A_i , then they will also be mutex for all *previous* levels at which they both appear. The same holds for mutexes between literals. It might not always appear that way in the figures, because the figures have a simplification: they display neither literals that cannot hold at level S_i nor actions that cannot be executed at level A_i . We can see that “mutexes decrease monotonically” is true if you consider that these invisible literals and actions are mutex with everything.

The proof is a little complex, but can be handled by cases: if actions A and B are mutex at level A_i , it must be because of one of the three types of mutex. The first two, inconsistent effects and interference, are properties of the actions themselves, so if the actions are mutex at A_i , they will be mutex at every level. The third case, competing needs, depends on conditions at level S_i : that level must contain a precondition of A that is mutex with a precondition of B . Now, these two preconditions can be mutex if they are negations of each other (in which case they would be mutex in every level) or if all actions for achieving one are mutex with all actions for achieving the other. But we already know that the available actions are increasing monotonically, so by induction, the mutexes must be decreasing.

Because the actions and literals increase and the mutexes decrease, and because there are only a finite number of actions and literals, every planning graph will eventually level off—all subsequent levels will be identical. Once a graph has leveled off, if it is missing one of the goals of the problem, or if two of the goals are mutex, then the problem can never be solved, and we can stop the GRAPHPLAN algorithm and return failure. If the graph levels off with all goals present and nonmutex, but EXTRACT-SOLUTION fails to find a solution, then we might have to extend the graph again a finite number of times, but eventually we can stop. This aspect of termination is more complex and is not covered here.